

5강

기본 자료형





1. 기본 자료형

- 자바는 정수, 실수, 논리값을 저장할 수 있는 기본타입을 제공한다.
- 자바가 제공하는 기본타입은 총8개이다.
- 변수가 사용할 공간의 크기와 특성에 따라 자료형을 사용하여 변수를 선언한다.
- **자바의 기본 연산 단위가 32비트(int)이기 때문에, 메모리 절약, 아주 큰 숫자가 없다면 정수는 int, 실수는 double을 표준으로 사용한다.**

구분	저장되는 값에 따른 분류	타입의 종류
기본 타입	정수 타입	byte, char, short, int, long
	실수 타입	float, double
	논리 타입	boolean F/T

➤ 정수 타입

자료형	타입	메모리 사용 크기		표현범위
문자	Char	2byte	16bit	0 ~ 65,535(유니코드) ← 숫자
정수형	Byte	1byte	8bit	-128 ~ 127
	Short	2byte	16bit	-32,768 ~ 32,767 } 직관적
	<u>Int</u>	4byte	32bit	-2,147,483,648 ~ 2,147,483,647
	Long	8byte	64bit	-9,223,372,036,845,775,808 ~ 9,223,372,036,845,775,807 ← 금용권

↙ 벗어나면 안됨

} 직관적

← 금용권



➤ 실수 타입, 논리 타입, String 타입

자료형	타입	메모리 사용 크기		표현범위
실수형	Float	4byte	0.0	-3.4E38 ~ +3.4E38
	double	8byte	0.0	-1.7E308 ~ +1.7E308
문자열	String	4byte		<u>객체 자료형</u>
논리형	Boolean	1bit	False	True, false

} 소수자리



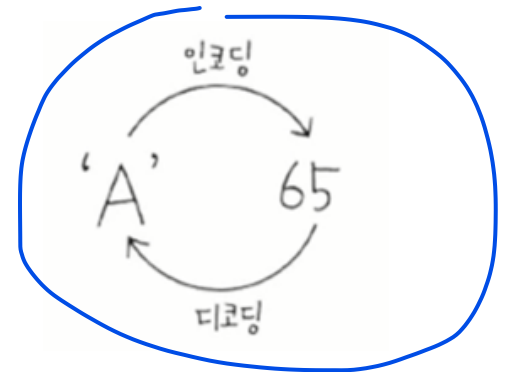
2. Char와 문자 인코딩 이해하기

➤ char

- 컴퓨터에서 문자는 사람처럼 그대로 이해되지 않는다.
- 컴퓨터는 모든 데이터를 0과 1로 이루어진 비트(bit)의 조합, 즉 숫자로 처리한다.
- 따라서 우리가 사용하는 문자 역시 내부적으로는 특정한 숫자 값으로 저장된다.
- 자바에서 문자를 저장할 때 사용하는 기본 자료형이 바로 **char**이다.
- char는 **문자 1개**를 저장하는 자료형이지만, 실제로는 문자에 대응되는 숫자 값(문자 코드)을 저장한다.
- 자바에서는 **char를 2바이트(16비트) 크기로 처리**하며, 이를 통해 다양한 문자를 표현할 수 있다.

➤ 인코딩과 디코딩

- **문자를 숫자로 바꾸는 과정을 인코딩(Encoding)**이라고 한다.
- 예를 들어 문자 'A'는 인코딩 과정을 거쳐 숫자 65로 저장된다.
- 반대로, 저장된 **숫자 값을 다시 문자로 바꾸는 과정을 디코딩(Decoding)**이라고 한다.
- 'A' → 65 : 인코딩
- 65 → 'A' : 디코딩
- 이와 같은 과정을 통해 컴퓨터는 문자를 처리한다.



= '
문장



3. 문자 세트(Character Set)

- 문자 세트란, 문자를 표현하기 위해 **각 문자에 어떤 숫자 값을 사용할지 미리 정해 놓은 규칙의 집합**이다.
- ASCII (아스키 코드)
 - 1바이트(8비트)로 문자를 표현 **영문 대소문자, 숫자, 특수문자** 등만 표현 가능
 - 예: 'A' → 65
- **Unicode (유니코드)**
 - 전 세계의 문자(한글, 중국어, 이모지 등)를 표현하기 위한 국제 표준**
 - 문자마다 고유한 코드 값을 가짐
 - 대표적인 인코딩 방식: UTF-8, UTF-16



4. char 타입 변수

```
package example;

public class DataType01 {

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        char ch='A';
        System.out.println(ch);
        System.out.println((int)ch);

        ch=66;
        System.out.println(ch);

        int ch2=67;
        System.out.println(ch2);
        System.out.println((char)ch2);
    }
}
```



Console X
<terminated> Dat
A
65
B
67
C



5. byte 타입 변수

- Byte 타입 변수에 허용 범위를 초과한 값을 대입했을 경우 컴파일 에러가 발생함을 주의하자!

```
package example;

public class DataType01 {

    public static void main(String[] args) {
        // TODO Auto-generated method stub

        byte var1= -128;
        byte var2= -30;
        byte var3= 0;
        byte var4= 30;
        byte var5= 127;
        byte var6= 128; // 컴파일 에러, 이유->byte 표현범위 -128~127
    }
}
```

Console × Problems @ Javadoc Declaration

<terminated> DataType01 [Java Application] C:\Program Files\Java\jdk-21\bin\javaw.exe (2025. 12. 22. 오전 3:16:08 - 오전 3:16:10 elapsed 0:00:01) [pid: 24636]

Exception in thread "main" java.lang.Error: Unresolved compilation problem:
Type mismatch: cannot convert from int to byte

at example.DataType01.main(DataType01.java:13)



6. long 타입 변수

- long 타입은 수치가 큰 데이터를 다루는 프로그램에 사용되는데 대표적인 예가 은행이나 과학관련 프로그램들이다.
- 기본적으로 컴파일러는 정수 리터럴을 int로 간주한다. 그래서 **정수 리터럴이 int 타입의 허용범위(-2,147,483,648 ~ 2,147,483,647)를 초과할 경우**, long 타입임을 컴파일러에게 알려 주어야 한다.
- 정수 리터럴 뒤에 소문자 l이나 대문자 L을 붙인다. 일반적으로 **대문자 L**을 사용한다.

```
package example;

public class DataType01 {

    public static void main(String[] args) {
        // TODO Auto-generated method stub

        long var6 = 1000000000000; //정수 int 타입의 허용범위를 초과해 컴파일 에러 출력
        long var7 = 1000000000000L;
        long var8 = 50;
    }
}
```

Console X Problems @ Javadoc Declaration

<terminated> DataType01 [Java Application] C:\Program Files\Java\jdk-21\bin\javaw.exe (2025. 12. 22. 오전 3:33:59 - 오전 3:34:00 elapsed 0:00:00) [pid: 4616]

Exception in thread "main" java.lang.Error: Unresolved compilation problem:
The literal 1000000000000 of type int is out of range

at example.DataType01.main(DataType01.java:8)



7. String 타입

- 작은 따옴표(")로 감싼 문자는 char타입 변수에 저장되어 유니코드로 저장되지만, 큰 따옴표("")로 감싼 문자 또는 여러 개의 문자들은 유니코드로 변환되지 않는다. 따라서 아래는 잘못된 코드이다.

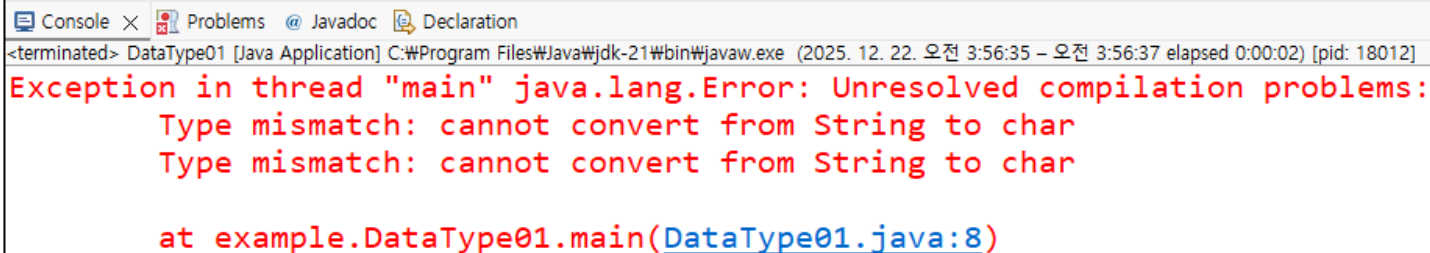
```
package example;

public class DataType01 {

    public static void main(String[] args) {
        // TODO Auto-generated method stub

        char var9 = "A";
        char var10 = "홍길동";
```

- 자바에서 큰따옴표("")로 감싼 문자들을 **문자열**이라 부른다. 작은 따옴표(")와 큰따옴표("")는 컴파일러가 문자 리터럴과 문자열 리터럴을 구별하는 기호로 사용된다.
- 문자열을 변수에 저장하고 싶으면 **String** 타입을 사용해야 한다.



```
Console × Problems @ Javadoc Declaration
<terminated> DataType01 [Java Application] C:\Program Files\Java\jdk-21\bin\javaw.exe (2025. 12. 22. 오전 3:56:35 - 오전 3:56:37 elapsed 0:00:02) [pid: 18012]
Exception in thread "main" java.lang.Error: Unresolved compilation problems:
    Type mismatch: cannot convert from String to char
    Type mismatch: cannot convert from String to char

    at example.DataType01.main(DataType01.java:8)
```



10. String 객체자료 형이란 ?

- 객체자료 형(Object)-4byte

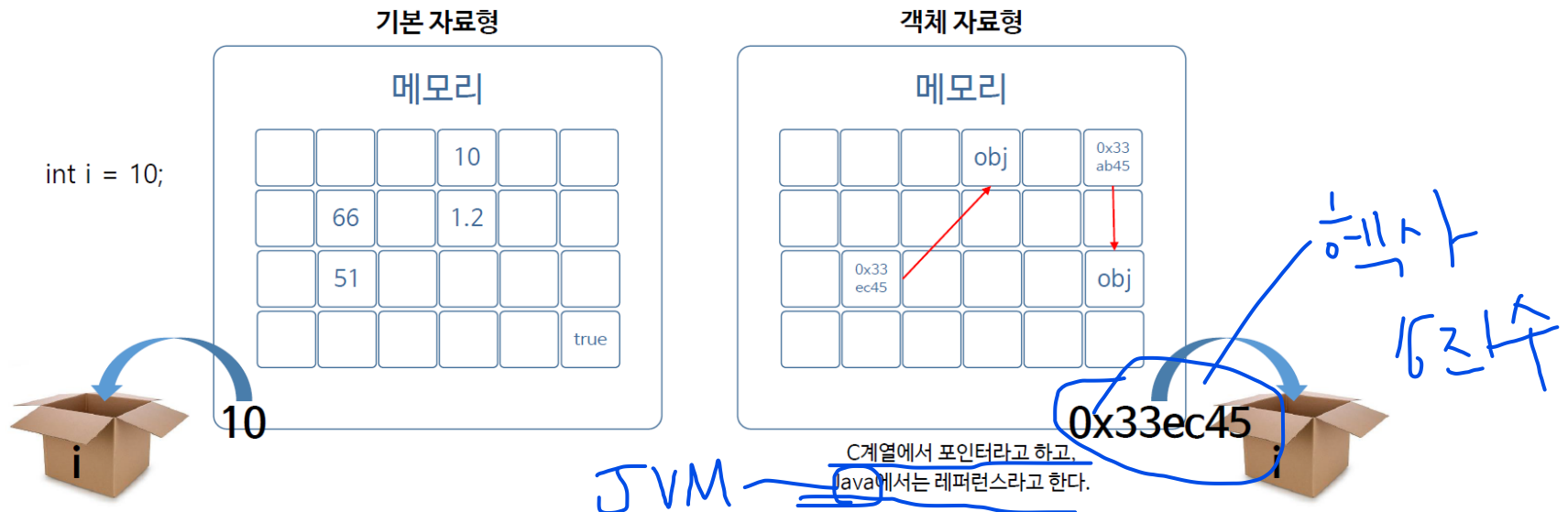
: 여러가지 데이터들이 모여 있는 복잡한 데이터로 기본 자료형에 비해 크기가 크다.

- (예. String, System, ArrayList 등등..)

* 기본자료 형은 제일 앞이 소문자로 시작!, 객체자료 형은 제일 앞이 대문자로 시작!

- String s = "ABC"; 이라는 값이 있을 때, String은 객체자료 형 이기 때문에 메모리에 "ABC"라는 값이 메모리 어딘가에 저장되고, 메모리에 할당된 s영역에는 "ABC"가 들어있는 주소 값이 들어간다.
- 기본 자료형은 데이터가 변수에 직접 저장되고, 객체자료 형은 객체 메모리 주소가 변수에 저장된다.

: int i = 10; 이라는 값이 있을 때, int는 기본 자료형이기 때문에 메모리에 할당된 i영역에 10이라는 값이 바로 들어간다.





7. String 타입 간단한 예제

```
String name="홍길동";
String job="프로그래머";
System.out.println(name);
// System.identityHashCode():
// String 객체변수의 주소는 아니지만 주소처럼 비교할수 있는 값
System.out.println(System.identityHashCode(name)); //10진수로 출력
int id = System.identityHashCode(name);
System.out.println(Integer.toHexString(id)); //16진수로 출력
System.out.println(job);
System.out.println(System.identityHashCode(job));
System.out.println();
```



홍길동
791452441
2f2c9b19
프로그래머
834600351



8. 이스케이프 문자란?

- 문자열 내부에 역슬래시(\)가 붙은 문자를 사용할 수 있는데, 이것을 **이스케이프 문자**라 한다. 이스케이프 문자를 사용하면 문자열 내부에 **특정 문자를 포함**시킬 수 있다.
- 이스케이프 문자를 사용하면 출력을 제어할 수 있다. 예를 들어 탭만큼 띄우거나(\t), 다음 줄로 개행(\n)을 지시할 수 있다.

```
package example;

public class DataType01 {

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        String str = "나는\"자바\"를 좋아합니다.";
        System.out.println(str);
    }
}
```



Console × Problems @ Javadoc
<terminated> DataType01 [Java Application]
나는"자바"를 좋아합니다.



9. 자주 사용하는 이스케이프 문자

이스케이프 문자	의미	사용 예
\n	줄바꿈	출력 줄 변경
\t	탭	표 형태 출력
\"	큰따옴표 출력	문자열 안 " "
'	작은따옴표 출력	char 설명
\\	역슬래시 출력	파일 경로

```
package example;

public class DataType01 {

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        System.out.println("번호\t이름\t직업");
        System.out.print("행 단위 출력 \n");
        System.out.print("행 단위 출력 \n");
        System.out.println("우리는 \"개발자\" 입니다.");
        System.out.print("봄\\여름\\가을\\겨울");
    }
}
```



Console × Problems @ Javadoc

<terminated> DataType01 [Java Application]

```
번호      이름      직업
행 단위 출력
행 단위 출력
우리는 "개발자" 입니다.
봄\여름\가을\겨울
```



11. 실수형 타입 - Float, Double

- 자바에서 double과 float은 실수(소수점이 있는 수)를 다루는 기본 데이터 타입이다.
- 실수는 기본 적으로 **double로 처리한다.** 금융 계산, 통계, 과학계산등 일반적인 실수 계산 대부분 사용
- Float 형으로 사용하는 경우 숫자에 f, F식별자를 명시 한다.
- float은 메모리를 아껴야 할 특별한 경우에만 사용한다.

➤ Double

- 64비트(8바이트) 실수 타입
- 소수점 약 15~16자리까지 정확
- 실수의 기본 타입 → 접미사 필요 없다.

```
package example;

public class VariableEx02 {

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        double d1=3.14;
        double d2=123.456789;

    }

}
```



11. 실수형 타입 - Float, Double

➤ Float

- 32비트(4바이트) 실수 타입
- 소수점 약 7자리 정도까지 정확
- 값 뒤에 **f 또는 F**를 반드시 붙여야 함

```
1 package example;
2
3 public class VariableEx02 {
4
5     public static void main(String[] args) {
6         // TODO Auto-generated method stub
7         float f1=3.14;
8         float f2=123.456789F;
9     }
10 }
```

에러 이유는 값 뒤에 F접미사를 입력하지 않아서

Problems @ Javadoc Declaration Console X

<terminated> VariableEx02 [Java Application] C:\Program Files\Java\jdk-21\bin\javaw.exe (2025. 12. 22. 오후 2:57:41 - 오후 2:57:44 elapsed 0:00:02) [pid: 11284]

Exception in thread "main" java.lang.Error: Unresolved compilation problem:
Type mismatch: cannot convert from double to float

at example.VariableEx02.main(VariableEx02.java:7)



12. 논리 타입 - Boolean

- 참(true) / 거짓(false) 두 값만 저장하는 데이터 타입이다.
- 조건 판단을 위한 논리 타입이다.
- 기본값은 false이다.

```
1 package example;
2
3 public class VariableEx02 {
4
5     public static void main(String[] args) {
6         // TODO Auto-generated method stub
7         boolean isLogin = false;
8         boolean result = true;
9     }
10
11 }
```


형 변환





1. 형 변환(type conversion)

- 데이터 자료형은 각각 사용하는 메모리 크기와 방식이 다르다.
- 서로 다른 자료형의 값이 대입되는 경우 형 변환이 일어난다.

➤ 묵시적 형 변환

- 작은 수에서 큰 수로
- 덜 정밀한 수에서 더 정밀한 수로 대입되는 경우
- 예) `long num = 3; // int 값에서 long으로 자동 형 변환, L, l의 접미사를 명시 하지 않아도 됨`

➤ 명시적 형 변환

- 묵시적 형 변환의 반대의 경우
- 반환 되는 자료 형을 명시해야 한다.
- 자료의 손실이 발생 할 수 있다.
- 예) `double dNum = 3.14;`
`int num = (int)dNum; //자료형 명시`



➤ 형 변환 간단한 예제

```
public static void main(String[] args) {  
    // 자동(묵시적) 형 변환 :  
    // 작은 공간의 메모리에서 큰 공간의 메모리로 이동  
    byte by = 10;  
    int in = by;  
    System.out.println("in = "+ in);  
  
    // 명시적 형 변환:  
    // 큰 공간의 메모리에서 작은 공간의 메모리로 이동  
    int ivar = 100;  
    byte bvar = (byte)ivar;  
    System.out.println("bvar =" +bvar);  
  
    ivar = 123456;  
    bvar = (byte)ivar;  
    System.out.println("bvar =" +bvar);  
}
```



```
Console × Problems  
<terminated> DataType01 [Java]  
in = 10  
bvar =100  
bvar =64
```



명시적 형 변환은 데이터가 누설 될 수 있다.

연습문제





➤ 여기서 잠깐 !

1. 다음 중 정수형 데이터 타입이 아닌 것은 무엇인가?

① byte

② short

③ int

④ float

2. 다음 중 컴파일 오류가 발생하는 코드는?

① int a = 10;

② double b = 3.14;

③ float c = 1.5;

④ long d = 100L;



➤ 여기서 잠깐 !

3. 다음 코드의 출력 결과는?

```
int a = 10;
```

```
int b = 3;
```

```
System.out.println(a / b);
```

① 3

② 3.3

③ 3.3333

④ 오류 발생

4. 다음 코드 중 정상적으로 실행되는 것은?

① float f = 3.14;

② float f = 3.14f;

③ float f = (float)"3.14";

④ float f = true;



➤ 여기서 잠깐 !

5. 다음 코드의 출력 결과는?

```
char ch = 'A';  
  
System.out.println(ch + 1);
```

① A

② B

③ 66

④ 오류 발생

6. 다음 중 기본 데이터 타입이 아닌 것은?

① int

② double

③ boolean

④ String



➤ 여기서 잠깐 !

7. 다음 코드의 출력 결과는?

```
int score = 70;  
  
boolean result = score >= 60;  
  
System.out.println(result);
```

- ① true
- ② false
- ③ 60
- ④ 오류 발생

8. 다음 중 반드시 오류가 발생하는 코드는?

- ① long a = 100000000000L;
- ② long b = 100;
- ③ long c = 1000000000000;
- ④ long d = 10L;



➤ 여기서 잠깐 !

9. 다음 코드의 출력 결과는?

```
System.out.println(10 + 20 + "30");
```

① 3030

② 102030

③ 303

④ 오류 발생

10. 다음 코드의 출력 결과는?

```
boolean a = true;
```

```
boolean b = false;
```

```
System.out.println(a && !b);
```

① true

② false

③ 오류 발생

④ 1